

Implementation Pro — Architecture

Version: 1.0

Last Updated: 2026-04-02

Owner: John Swapp

Status: Active — Living Document

AI agents that run implementations. You run the show.

Implementation Pro

Implementation Pro — Architecture & Development Plan

Version: 1.0

Last Updated: 2026-04-02

Owner: John Swapp

Status: Active — Living Document

1. Architecture Overview

Implementation Pro is a multi-tenant, mobile-first SaaS platform built on a serverless architecture. The system is organized into three operational layers (Operations Automation, Delivery Governance, Work Execution) on top of a core engagement management platform, with two cross-cutting capabilities (Self-Learning, Self-Healing).

System Context



Design Principles

- Server-first rendering. Server Components are the default. Client Components (`"use client"`) are only added at the leaf level when interactivity, browser APIs, or hooks are required.

- **Separation of concerns.** Each operational layer has its own database tables, API routes, and UI components. Layers communicate through the database, not through direct function calls.
- **Schema-driven contracts.** Zod schemas define the validation contract for all forms and API inputs. TypeScript types are inferred from schemas, never hand-authored separately.
- **Org-scoped data isolation.** Every data table includes an `org_id` foreign key. Row Level Security (RLS) policies enforce that users can only access data within their organization.
- **Mobile-first design.** All UI is designed for 375px viewports first, then enhanced for larger screens. Touch targets are minimum 44px.
- **Graceful degradation.** If an external service (AI, email, billing) is unavailable, the platform's core engagement management features remain functional.

2. Technology Stack

Decided and In Use

Layer	Technology	Version	Purpose
Framework	Next.js	16.2.1	App Router, Server Components, Route Handlers, SSR
Language	TypeScript	6.0.2	Strict mode, ES2017 target, bundler module resolution
UI Library	React	18.3.1	Component rendering
Styling	Tailwind CSS	3.4.17	Utility-first CSS, dark mode support (class-based)
Component Variants	class-variance-authority	0.7.1	Variant-driven component styling
Class Merging	tailwind-merge	3.5.0	Conflict-free class name composition via <code>cn()</code>
Icons	lucide-react	1.7.0	Consistent icon set
State (Server)	TanStack React Query	5.95.2	Cache, sync, and update server state in Client Components
State (UI)	React built-in hooks	—	Local UI state (no external state management library)
Forms	react-hook-form	7.72.0	Form state management

Layer	Technology	Version	Purpose
Validation	zod	4.3.6	Schema validation + type inference via @hookform/resolvers (5.2.2)
Database	Supabase (PostgreSQL)	supabase-js 2.101.0, SSR 0.10.0	Multi-tenant database, auth, realtime, storage
AI (Primary)	Anthropic Claude API	SDK 0.82.0	Document generation, risk analysis, agent execution
AI (Fallback)	OpenAI	Planned	Redundancy for AI operations
Billing	Stripe	21.0.1	Subscription management, checkout, customer portal
Email	Resend	6.10.0	Transactional email (client updates, invitations)
Deployment	Vercel	—	Serverless hosting, edge CDN, preview deployments
E2E Testing	Playwright	1.59.1	Cross-browser testing (chromium + mobile-chrome)
Package Manager	npm	—	Only permitted package manager; no yarn/pnpm/bun

Not Permitted (Explicit Exclusions)

What	Why
Redux, Zustand, Jotai	React Query handles server state; React hooks handle UI state
Additional CSS frameworks	Tailwind is the sole styling system
Prisma, Drizzle (ORMs)	Supabase client is the data layer
Jest, Vitest	Playwright E2E is the sole test runner
yarn, pnpm, bun	npm only, package-lock.json is the lock file

3. Application Architecture

3.1 Directory Structure

```

SaaS_IMP-Playbook_Items/
  ■■■■ src/
  ■ ■■■■ app/ # Next.js App Router
  ■ ■ ■■■■ (auth)/ # Auth route group
  ■ ■ ■ ■■■■ callback/route.ts # OAuth callback handler
  ■ ■ ■ ■■■■ login/page.tsx # Login page
  ■ ■ ■ ■■■■ signup/page.tsx # Signup page
  ■ ■ ■■■■ (dashboard)/ # Authenticated app routes
  ■ ■ ■ ■■■■ layout.tsx # Dashboard shell (sidebar + nav)
  ■ ■ ■ ■■■■ dashboard/page.tsx # Overview dashboard
  ■ ■ ■ ■■■■ engagements/ # Engagement list + detail
  ■ ■ ■ ■■■■ page.tsx # Engagement list view
  ■ ■ ■ ■■■■ new/page.tsx # Create engagement form
  ■ ■ ■ ■■■■ [id]/page.tsx # Engagement detail (tabbed)
  ■ ■ ■ ■■■■ agents/page.tsx # AI agent console
  ■ ■ ■ ■■■■ governance/page.tsx # Risk + delivery governance
  ■ ■ ■ ■■■■ intelligence/page.tsx # Cross-engagement analytics
  ■ ■ ■ ■■■■ onboarding/page.tsx # First-time org setup
  ■ ■ ■ ■■■■ settings/page.tsx # Org + profile settings
  ■ ■ ■■■■ (marketing)/ # Public pages (no auth required)
  ■ ■ ■■■■ api/ # Route Handlers (REST endpoints)
  ■ ■ ■ ■■■■ agents/execute/route.ts
  ■ ■ ■ ■■■■ email/send/route.ts
  ■ ■ ■ ■■■■ engagements/create/route.ts
  ■ ■ ■ ■■■■ org/create/route.ts
  ■ ■ ■ ■■■■ seed/route.ts
  ■ ■ ■ ■■■■ stripe/
  ■ ■ ■ ■■■■ checkout/route.ts
  ■ ■ ■ ■■■■ portal/route.ts
  ■ ■ ■ ■■■■ webhook/route.ts
  ■ ■ ■ ■■■■ waitlist/route.ts
  ■ ■ ■■■■ layout.tsx # Root layout (html, body, providers)
  ■ ■ ■■■■ page.tsx # Landing page (public)
  ■ ■ ■■■■ providers.tsx # React Query provider wrapper
  ■ ■ ■■■■ error.tsx # Global error boundary
  ■ ■ ■■■■ not-found.tsx # 404 page
  ■ ■■■■ components/
  ■ ■ ■■■■ engagements/ # Engagement-specific UI
  ■ ■ ■ ■■■■ engagement-header.tsx
  ■ ■ ■ ■■■■ engagement-tabs.tsx
  ■ ■ ■ ■■■■ tabs/ # 16 tab components
  ■ ■ ■ ■■■■ agents-tab.tsx
  ■ ■ ■ ■■■■ budget-tab.tsx
  ■ ■ ■ ■■■■ checklist-tab.tsx
  ■ ■ ■ ■■■■ client-updates-tab.tsx
  ■ ■ ■ ■■■■ decisions-tab.tsx
  ■ ■ ■ ■■■■ delivery-trends-tab.tsx
  ■ ■ ■ ■■■■ health-tab.tsx
  ■ ■ ■ ■■■■ lessons-tab.tsx
  ■ ■ ■ ■■■■ project-plan-tab.tsx
  ■ ■ ■ ■■■■ raci-tab.tsx
  ■ ■ ■ ■■■■ reports-tab.tsx
  ■ ■ ■ ■■■■ resources-tab.tsx

```

```

■ ■ ■ ■ ■ risk-signals-tab.tsx
■ ■ ■ ■ ■ scope-tab.tsx
■ ■ ■ ■ ■ stakeholders-tab.tsx
■ ■ ■ ■ ■ time-entries-tab.tsx
■ ■ ■ ■ ■ landing/ # Marketing page components
■ ■ ■ ■ ■ waitlist-form.tsx
■ ■ ■ ■ ■ layout/ # Shell components
■ ■ ■ ■ ■ dashboard-shell.tsx
■ ■ ■ ■ ■ mobile-nav.tsx
■ ■ ■ ■ ■ sidebar.tsx
■ ■ ■ ■ ■ settings/
■ ■ ■ ■ ■ compliance-rules-section.tsx
■ ■ ■ ■ ■ hooks/
■ ■ ■ ■ ■ use-org.tsx # Organization context hook
■ ■ ■ ■ ■ lib/
■ ■ ■ ■ ■ ai/
■ ■ ■ ■ ■ client.ts # Anthropic SDK client
■ ■ ■ ■ ■ prompts.ts # Agent system prompts
■ ■ ■ ■ ■ email/
■ ■ ■ ■ ■ client.ts # Resend client
■ ■ ■ ■ ■ templates.ts # Email templates
■ ■ ■ ■ ■ stripe/
■ ■ ■ ■ ■ client.ts # Stripe client
■ ■ ■ ■ ■ supabase/
■ ■ ■ ■ ■ client.ts # Browser client (Client Components)
■ ■ ■ ■ ■ server.ts # Server client (Server Components + Route Handlers)
■ ■ ■ ■ ■ utils.ts # cn() utility + shared helpers
■ ■ ■ ■ ■ styles/
■ ■ ■ ■ ■ globals.css # Tailwind base + custom CSS
■ ■ ■ ■ ■ types/
■ ■ ■ ■ ■ database.types.ts # Auto-generated Supabase types
■ ■ ■ ■ ■ index.ts # Hand-authored domain types
■ ■ ■ ■ ■ supabase/
■ ■ ■ ■ ■ migrations/
■ ■ ■ ■ ■ 001_initial_schema.sql # 846-line baseline (33 tables, RLS, triggers, seed)
■ ■ ■ ■ ■ seed/
■ ■ ■ ■ ■ acme_corp.sql # Standard dev/test seed data
■ ■ ■ ■ ■ e2e/ # Playwright E2E tests
■ ■ ■ ■ ■ [config files] # package.json, tsconfig.json, next.config.js, etc.

```

3.2 Data Flow Patterns

Server Component Data Fetching:

```

Browser Request
→ Vercel Edge / Server
→ Next.js Server Component
→ createServerClient() from @supabase/ssr
→ Supabase PostgreSQL (RLS enforced)
→ Return rendered HTML to browser

```

Client Component Mutations:

```

User Action in Browser
→ Client Component (with "use client")

```

```

→ React Query useMutation()
→ POST to Route Handler (src/app/api/*)
→ createServerClient() from @supabase/ssr
→ Supabase PostgreSQL (RLS enforced)
→ Return { data, error } response
→ onSuccess: invalidateQueries() for cache refresh

```

AI Agent Execution:

```

User triggers agent (or scheduled trigger)
→ POST /api/agents/execute
→ Load agent_definition (system_prompt, I/O schema)
→ Build context from engagement data
→ Call Anthropic Claude API
→ Store execution log (tokens, duration, cost)
→ Store artifact (document, report, plan)
→ If propose-and-wait: return for user review
→ If auto-execute: apply output directly

```

Stripe Billing:

```

User selects plan → POST /api/stripe/checkout → Stripe Checkout Session
→ User completes payment on Stripe-hosted page
→ Stripe sends webhook → POST /api/stripe/webhook
→ Update organization.plan in Supabase
→ Confirm trial_ends_at or subscription status

User manages subscription → POST /api/stripe/portal → Stripe Customer Portal URL

```

3.3 Authentication Architecture

Supabase Auth handles all authentication with SSR-aware session management:

- Sign-up/Login: Email + password or OAuth (configurable per Supabase project)
- Session management: Cookie-based sessions via @supabase/ssr middleware
- Route protection: Dashboard route group (dashboard) requires authenticated session; layout.tsx checks auth state and redirects unauthenticated users to /login
- OAuth callback: /callback/route.ts exchanges auth code for session
- Profile auto-creation: Database trigger on_auth_user_created creates a profiles row when a new user signs up

3.4 Multi-Tenancy Model

- Every data table (except profiles, agent_definitions, system_health_checks, error_patterns) has an org_id column

- RLS policies use a helper function `is_org_member(org_id)` that checks the authenticated user's membership in the organization
- Organization roles (owner, admin, member, viewer) provide granular access control
- Admin-only operations (org settings, member management) have additional RLS policies checking the `role` column

3.5 Database Schema Architecture

The schema is organized into six domains with 33 tables total:

- Foundation (13 tables):** Core platform — users, orgs, engagements, scope, stakeholders, decisions, RACI, checklists, lessons, reports, activity log, automation rules
 - Layer 1 — Operations (6 tables):** Resource profiles, allocations, time entries, financial tracking, compliance rules, compliance violations
 - Layer 2 — Governance (4 tables):** Project plans, risk signals, client updates, delivery signals
 - Layer 3 — Agents (4 tables):** Agent definitions (6 seeded system agents), agent tasks, agent executions, agent artifacts
 - Self-Learning (3 tables):** Learning feedback, org learning context, outcome correlations
 - Self-Healing (3 tables):** System health checks, self-healing events, error patterns
- 41 custom PostgreSQL enum types enforce controlled vocabularies across the schema. All enums are defined in the baseline migration (`001_initial_schema.sql`) and new values are added via `ALTER TYPE ... ADD VALUE` in subsequent migrations.
- Indexing strategy:** 41 indexes cover primary query patterns — org-scoped lookups, engagement-scoped lookups, status filters, and time-series queries.

4. API Architecture

4.1 Route Handlers (Current)

Route	Method	Purpose
<code>/api/agents/execute</code>	POST	Trigger AI agent execution for an engagement
<code>/api/email/send</code>	POST	Send transactional email (client updates, invites)
<code>/api/engagements/create</code>	POST	Create a new engagement
<code>/api/org/create</code>	POST	Create a new organization

Route	Method	Purpose
/api/seed	POST	Seed development data (Acme Corp scenario)
/api/stripe/checkout	POST	Create Stripe Checkout session
/api/stripe/portal	POST	Generate Stripe Customer Portal URL
/api/stripe/webhook	POST	Handle Stripe webhook events
/api/waitlist	POST	Add email to pre-launch waitlist

4.2 API Design Conventions

- All Route Handlers export named HTTP method functions (GET, POST, PUT, DELETE)
- Request validation via Zod schemas
- Response format: { data: T, error: null } on success, { data: null, error: string } on failure
- Error handling: try/catch around all external service calls (Supabase, Stripe, Anthropic)
- Server-side error logging; safe error messages returned to client

4.3 API Routes Needed (Not Yet Built)

Route	Method	Purpose	Phase
/api/engagements/[id]	GET, PUT, DELETE	Engagement CRUD	Phase 1
/api/engagements/[id]/scope	GET, POST, PUT, DELETE	Scope item management	Phase 2
/api/engagements/[id]/stakeholders	GET, POST, PUT, DELETE	Stakeholder management	Phase 2
/api/engagements/[id]/raci	GET, POST, PUT, DELETE	RACI matrix management	Phase 2

Route	Method	Purpose	Phase
<code>/api/engagements/[id]/checklist</code>	GET, POST, PUT	Checklist management	Phase 2
<code>/api/engagements/[id]/reports</code>	GET, POST	Status report generation	Phase 2
<code>/api/engagements/[id]/time-entries</code>	GET, POST, PUT	Time entry tracking	Phase 3
<code>/api/engagements/[id]/financials</code>	GET, PUT	Financial tracking	Phase 3
<code>/api/engagements/[id]/risks</code>	GET	Risk signal data	Phase 4
<code>/api/engagements/[id]/client-updates</code>	GET, POST, PUT	Client update management	Phase 4
<code>/api/agents/[id]/tasks</code>	GET	Agent task history	Phase 5
<code>/api/agents/[id]/feedback</code>	POST	Submit feedback on agent output	Phase 6

5. Frontend Architecture

5.1 Component Hierarchy

```

RootLayout (layout.tsx)
  ■■■ Providers (providers.tsx) – React Query
  ■■■ (marketing) – Public pages
  ■ ■■■ LandingPage – WaitlistForm
  ■■■ (auth) – Login, Signup
  ■■■ (dashboard) – DashboardLayout (sidebar + mobile nav)
  ■■■ Dashboard – Overview cards, engagement summary
  ■■■ Engagements – List view
  ■ ■■■ NewEngagement – Create form
  ■ ■■■ EngagementDetail – EngagementHeader + EngagementTabs
  ■ ■■■ Tabs: Scope, Stakeholders, RACI, Checklist,
  ■ Decisions, Health, Budget, Resources, TimeEntries,
  ■ ProjectPlan, RiskSignals, ClientUpdates,

```

- DeliveryTrends, Lessons, Reports, Agents
- Agents – Agent console
- Governance – Risk + delivery governance
- Intelligence – Cross-engagement analytics
- Onboarding – First-time org setup
- Settings – Org + profile settings
- ComplianceRulesSection

5.2 Component Conventions

- One component per file, named export matching filename
- `<font name="Courier" color="#334155">class-variance-authority</font>` (cva) for variant-driven styling
- `<font name="Courier" color="#334155">cn()</font>` from `<font name="Courier" color="#334155">@/lib/utls</font>` for class merging (tailwind-merge)
- Co-located types for component-specific props; shared types in `<font name="Courier" color="#334155">src/types</font>`
- No default exports except for pages and layouts (Next.js requirement)

5.3 State Management Strategy

State Type	Solution	Example
Server state (cached)	TanStack React Query	Engagement list, stakeholder data
Server state (mutations)	React Query <code>useMutation</code>	Create engagement, update scope item
UI state (local)	React <code>useState</code> / <code>useReducer</code>	Tab selection, form visibility, modal open/close
Auth state	Supabase SSR session	Checked in Server Components and middleware
Org context	Custom <code>useOrg</code> hook	Current org ID, role, settings

5.4 Mobile-First Design Requirements

- All layouts designed for 375px width first
- Progressive enhancement for tablet (768px) and desktop (1024px+)
- Touch targets minimum 44px × 44px
- No horizontal scroll on any viewport
- Bottom navigation or hamburger menu for mobile
- Sidebar collapses to overlay on mobile (existing `<font name="Courier" color="#334155">mobile-nav.tsx</font>`)

6. External Service Integration

6.1 Supabase

Client instances:

- `src/lib/supabase/client.ts` — Browser client via `createBrowserClient()` from `@supabase/ssr`. Used in Client Components.
- `src/lib/supabase/server.ts` — Server client via `createServerClient()` from `@supabase/ssr`. Used in Server Components and Route Handlers.

Configuration:

- `NEXT_PUBLIC_SUPABASE_URL` — Project URL (public, used client-side)
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` — Anon key (public, RLS-restricted)
- `SUPABASE_SERVICE_ROLE_KEY` — Service role key (server-side only, bypasses RLS)

6.2 Anthropic Claude API

Client: `src/lib/ai/client.ts` — Anthropic SDK instance

Prompts: `src/lib/ai/prompts.ts` — Agent system prompts (also seeded in `agent_definitions` table)

Configuration: `ANTHROPIC_API_KEY` (server-side only)

Model: `claude-sonnet-4-20250514` (default, stored in `agent_executions.model_used`)

6.3 Stripe

Client: `src/lib/stripe/client.ts` — Stripe SDK instance

Routes:

- Checkout: Creates a Stripe Checkout Session with plan pricing
- Portal: Generates a Customer Portal URL for self-service billing
- Webhook: Handles `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`

Configuration:

- `STRIPE_SECRET_KEY` (server-side only)
- `STRIPE_WEBHOOK_SECRET` (server-side only, for webhook signature verification)
- `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` (client-side)

6.4 Resend

Client: src/lib/email/client.ts — Resend SDK instance

Templates: src/lib/email/templates.ts — Email templates for client updates, invitations

Configuration: RESEND_API_KEY (server-side only)

7. Testing Strategy

7.1 Test Stack

Playwright E2E is the sole configured test runner. No unit test framework is installed.

Configuration: playwright.config.ts at root

Test directory: e2e/

Projects:

- chromium — Desktop Chrome
- mobile-chrome — Pixel 5 emulation (mobile viewport)

7.2 Mandatory 5-Phase Protocol

Every code change completes all five phases before being marked done:

- Static Analysis: npm run lint + npx tsc --noEmit — zero errors/warnings
- Build Verification: npm run build — clean build, zero warnings
- E2E Tests: npx playwright test — full suite passes
- Functional Testing: Manual testing on desktop + mobile (375px), covering empty states, error states, loading states, auth states, double-submit
- Regression Check: Manual verification of 3 core flows not touched by the change

7.3 Self-Healing Loop

Any failure in the testing protocol triggers:

```
WHILE (any_check_fails) {
  1. DIAGNOSE → Read the actual error
  2. FIX → Minimal targeted fix
  3. RESTART → Back to Phase 1 (full protocol)
  4. LOG → Append entry to Self-Learning.md
}
```

Maximum 3 iterations before stopping and reassessing the approach.

8. Deployment Architecture

8.1 Environments

Environment	Platform	Purpose
Local Dev	<code>npm run dev</code> (localhost:3000)	Development and manual testing
Preview	Vercel Preview Deployments	PR-level preview URLs
Production	Vercel Production	Live application

8.2 Environment Variables

Variable	Scope	Required
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Client + Server	Yes
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Client + Server	Yes
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Server only	Yes
<code>ANTHROPIC_API_KEY</code>	Server only	Yes (for AI features)
<code>STRIPE_SECRET_KEY</code>	Server only	Yes (for billing)
<code>STRIPE_WEBHOOK_SECRET</code>	Server only	Yes (for billing)
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code>	Client	Yes (for billing)
<code>RESEND_API_KEY</code>	Server only	Yes (for email)

8.3 CI/CD

Currently manual deployment via `<font name="Courier" color="#334155">npx vercel</font>`. Planned: automated deployments on push to `<font name="Courier" color="#334155">main</font>` via Vercel Git integration.

Pre-deployment checklist:

- All 5 testing phases pass
- `npm run build` succeeds locally
- No `.env` files in the commit
- Feature branch merged to `main` via PR

9. Development Plan

Phase 1: Foundation (Weeks 1-6)

Goal: Working auth, org management, and engagement CRUD with mobile-first UI.

Week	Deliverable	Files Touched
1-2	Supabase project setup, migration applied, auth flow working (signup → login → dashboard → logout)	<code>src/app/(auth)/</code> , <code>src/lib/supabase/</code> , <code>.env.local</code>
2-3	Organization CRUD + invite system, <code>useOrg</code> hook connected to real data	<code>src/app/api/org/</code> , <code>src/hooks/use-org.tsx</code> , <code>src/app/(dashboard)/onboarding/</code>
3-4	Engagement list view (table/kanban toggle), create form, detail page shell	<code>src/app/(dashboard)/engagements/</code> , <code>src/components/engagements/</code>
4-5	Mobile-first responsive pass on all views, bottom nav for mobile, sidebar collapse	<code>src/components/layout/*</code> , all page components
5-6	Dashboard overview (engagement count, health distribution, recent activity), E2E tests	<code>src/app/(dashboard)/dashboard/*</code> , <code>e2e/</code>

Exit criteria: User can sign up, create an org, create engagements, view dashboard — all working on mobile and desktop with passing E2E tests.

Phase 2: Templates & Data Entry (Weeks 7-12)

Goal: All engagement tabs have functional CRUD for their data domains.

Week	Deliverable	Tables Used
7-8	Scope tab: MoSCoW tracker with add/edit/reorder	scope_items
8-9	Stakeholder tab + Decision log tab	stakeholders, decisions
9-10	RACI matrix tab (editable grid) + Checklist tab (kickoff + go-live)	raci_items, checklist_items
10-11	Lessons learned tab + Status reports tab (generate from engagement data)	lessons_learned, status_reports
11-12	Mobile optimization pass on all tabs, E2E tests for each tab	All above

Exit criteria: Every engagement tab is functional with real Supabase data. User can manage a full engagement lifecycle from kickoff to go-live.

Phase 3: L1 — Operations Automation (Weeks 13-18)

Goal: Resource management, time tracking, financial tracking, and compliance engine.

Week	Deliverable	Tables Used
13-14	Resource profiles + allocation per engagement	resource_profiles, resource_allocations
14-15	Time entry tracking with billable/non-billable categories	time_entries
15-16	Budget tab: financial tracking dashboard per engagement	financial_tracking
16-17	Compliance rule engine + violation tracking	compliance_rules, compliance_violations
17-18	Settings page: compliance rules configuration UI	compliance_rules, src/components/settings/*

Exit criteria: Director persona can manage team resources, track time and financials, and configure compliance rules — all with mobile-responsive UI.

Phase 4: L2 — Delivery Governance (Weeks 19-26)

Goal: AI-powered risk detection, health scoring, and automated client communications.

Week	Deliverable	Tables Used
19-20	Project plan tab: AI-generated project plans from engagement context	project_plans
21-22	Risk signals tab: detection engine, severity scoring, evidence tracking	risk_signals
22-23	Delivery signals: velocity, scope drift, stakeholder engagement metrics	delivery_signals
23-24	Health score computation from delivery signals	engagements.health_score
24-25	Client updates: AI-drafted updates with approval workflow	client_updates
25-26	Governance dashboard: cross-engagement risk posture	src/app/(dashboard)/governance/*

Exit criteria: AI generates project plans and risk assessments from engagement data. Client updates can be drafted, approved, and sent. Health scores are computed automatically.

Phase 5: L3 — AI Agents (Weeks 27-34)

Goal: Six AI agents operational with task queue, execution logging, and artifact management.

Week	Deliverable	Tables Used
27-28	Agent execution pipeline: queue → run → complete/fail, with token/cost tracking	agent_tasks, agent_executions
28-29	Documentation Agent + Communication Agent (propose-and-wait)	agent_definitions, agent_artifacts
30-31	Testing Agent + Migration Agent	agent_definitions, agent_artifacts

Week	Deliverable	Tables Used
31-32	Configuration Agent + Analysis Agent	<code>agent_definitions, agent_artifacts</code>
32-33	Agent console UI: trigger, monitor, review agent work	<code>src/app/(dashboard)/agents/*</code>
33-34	Artifact management: draft → approved → delivered → archived	<code>agent_artifacts</code>

Phase 6: Self-Learning (Weeks 27-34, parallel with Phase 5)

Goal: Feedback pipeline that makes agents smarter over time.

Week	Deliverable	Tables Used
27-29	Feedback capture on every agent output (accepted/modified/rejected)	<code>learning_feedback</code>
29-31	Org learning context: auto-extract patterns from feedback	<code>org_learning_context</code>
31-33	Outcome correlations: signal → action → result tracking	<code>outcome_correlations</code>
33-34	Prompt refinement: inject org context into agent system prompts	<code>agent_definitions, org_learning_context</code>

Phase 7: Self-Healing (Weeks 27-34, parallel with Phase 5)

Goal: Automated health monitoring and failure recovery.

Week	Deliverable	Tables Used
27-29	System health checks: API availability, DB perf, agent success rate	<code>system_health_checks</code>
29-31	Self-healing events: auto-retry, circuit break, fallback activation	<code>self_healing_events</code>
31-34	Error pattern recognition + auto-resolution	<code>error_patterns</code>

10. Migration Strategy

Database Migrations

- Baseline: `001_initial_schema.sql` (846 lines, deployed)
- All changes go in new numbered files: `002_<i>.sql</i>`, `003_</i>.sql</code>, etc.`
- Every migration is idempotent (`IF NOT EXISTS</code>, <code>IF EXISTS</code>, <code>OR REPLACE</code>)`
- Every migration includes a rollback comment block
- After any schema change: regenerate `src/types/supabase.ts` via `npx supabase gen types</code>`

Gumroad to Stripe Migration

Gumroad is being dropped entirely. All billing moves to Stripe:

- New customers go through Stripe Checkout from day one
- Existing Gumroad subscribers (SaaS Implementation Playbook Kit at \$12/mo or \$79/yr) need a migration path — **this requires a migration plan (not yet defined)**
- Stripe handles subscription management, customer portal, and webhook-driven plan updates

11. Security Architecture

Authentication & Authorization

- Supabase Auth for identity management (email/password + OAuth)
- Cookie-based sessions via `@supabase/ssr</code>`
- RLS on all tables — no data access without authenticated session and org membership
- Admin-only operations gated by `role</code> column in org_members</code>`

Data Protection

- Data encrypted at rest (Supabase/AWS default: AES-256)
- Data encrypted in transit (TLS 1.3)
- API keys in environment variables only — never committed to code
- `.env.local</code> and <code>.env*.local</code> in .gitignore</code>`
- Service role key used only server-side for system operations (health checks, error patterns)

API Security

- Route Handlers validate input via Zod schemas

- Stripe webhooks verified via `STRIPE_WEBHOOK_SECRET` signature check
- Server Actions body size limited to 2MB (`next.config.js`)
- No direct database access from Client Components — all mutations through Route Handlers or Server Actions

12. Monitoring & Observability (Planned)

What	Tool	Status
Error tracking	Sentry	Planned
Performance monitoring	Vercel Analytics	Planned
AI agent metrics	Custom (system_health_checks table)	Schema exists
Uptime monitoring	Supabase built-in	Available
Log aggregation	Vercel Logs	Available

13. Known Technical Debt & Risks

Item	Type	Impact	Phase to Address
No unit tests (Playwright E2E only)	Testing gap	Medium — utility functions and data transformations untested	Phase 2+
pgvector disabled	Feature gap	Low until self-learning semantic search is needed	Phase 6
<code>db:migrate</code> script is a no-op	DX friction	Low — migrations applied manually via SQL editor	Phase 1
No automated CI/CD pipeline	Process gap	Medium — manual deployments risk human error	Phase 1
OpenAI fallback not implemented	Reliability gap	Medium — Anthropic API outage would disable all AI features	Phase 5
No rate limiting on API routes	Security gap	Medium — potential for abuse	Phase 3

Item	Type	Impact	Phase to Address
Server Actions body limit 2MB	Constraint	Low — sufficient for current use cases	Monitor

<i>This document is the source of truth for how Implementation Pro is built. Updated alongside the codebase.</i>